Designing, Building and Verifying an Embedded System

Introduction to ARM Cortex-M Microprocessors — Chapter 14

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

Why This Chapter Matters

Earlier chapters used one peripheral at a time. A real system samples on a schedule, answers buttons, drives indicators and reports to a host, all at once, on one processor, with no operating system. The hard part is deciding *before* coding which peripherals the job needs and whether the timing can work. Then you must *prove* the result meets the requirement.

After this chapter, you can:

1. Turn a requirement into a decomposition and a peripheral choice.
2. Choose polling, timed polling or interrupts per activity.
3. Structure a cooperative, non-blocking main loop.
4. Protect data shared between ISR and main.
5. Write a timing budget, then verify it by measurement.
6. Diagnose a multi-peripheral system bottom-up.

© Muním Zabidi

By the end of this chapter you will know how one program does all of this together, with no task blocking another:

```
Sensor Monitor ready.
Level: 1650 mV          // from the potentiometer on PA0
-> raw                  // button on PC13 toggled the mode
Raw:   2048             // LED bar on PB0-PB3 shows the level
```

Questions this code raises, and where this chapter answers them:

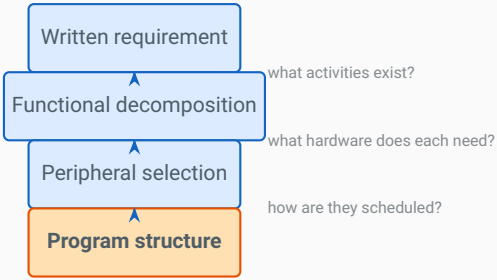
Question raised	Answered in
How can it do three jobs at once?	The non-blocking main loop
Why does the sample rate not drift?	Design decisions worth noting
Can I prove it meets its deadline?	The timing budget
What if it does not work?	A debugging methodology

Table of Contents

1. Why This Matters
2. From Requirement to Architecture
3. The Project: A Sensor Monitor
4. Architecture: The Non-Blocking Main Loop
5. The Complete Program
6. Design Decisions Worth Noting
7. The Timing Budget
8. Predict, Then Measure
9. What a Working Demonstration Does Not Prove
10. A Debugging Methodology
11. What You Have Learned
12. The Capstone Project
13. Summary

The Step Before Any Code

A requirement is prose. An architecture is a set of decisions. Getting from one to the other is the engineering, and it happens on paper.



Why this order matters

Choosing peripherals before decomposing the activities means you discover the pin conflict after the board is made.

Four Ways to Structure an Activity

Structure	Use when	Cost
Blocking poll	Never, in a real system	Uses all the CPU time; blocks everything else
Timed poll	The activity has a known period and loose jitter	Simple; jitter is the loop time
Interrupt	The event is external and the deadline is tight	Needs care with shared data
DMA	Bulk data, no per-byte decision	Setup cost; fewest CPU cycles

The decision is per activity, not per program

One system routinely uses all four: DMA for an ADC stream, an interrupt for a button, a timed poll for a display refresh, and the main loop for everything with no deadline at all.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 **The Project: A Sensor Monitor**

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

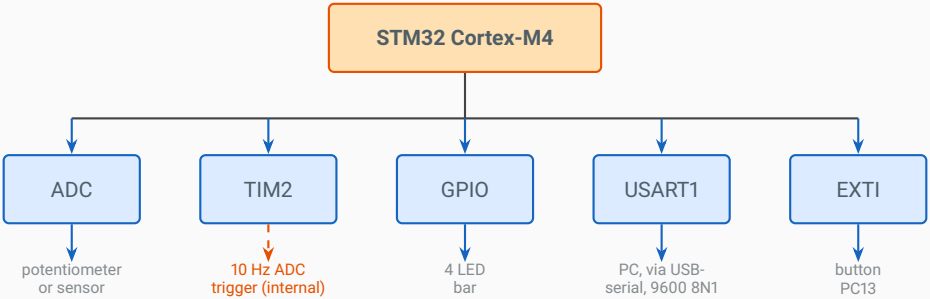
10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

The Project



Five peripherals, one processor, no OS.

1. Sample the ADC at exactly 10 Hz.
2. Display the level as a 4-LED bar.
3. Transmit each reading as text over USART1 (PB6/PB7, USB-serial adapter).
4. Button press toggles report between raw counts and millivolts.
5. Never miss a sample. No interrupt handler may block.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 **Architecture: The Non-Blocking Main Loop**

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

The Pattern

```
while (1) {
    uint32_t now = millis();

    if (now - last_sensor >= 2000) {    // every 2 s
        last_sensor = now;
        read_sensor();
    }
    if (now - last_display >= 100) {    // every 100 ms
        last_display = now;
        update_display();
    }
    if (button_flag) {                  // set by an ISR
        button_flag = 0;
        handle_button();
    }
}
```

Nothing waits

Each activity asks “is it my turn yet?” and returns immediately if not. The loop runs thousands of times per second, and every activity gets serviced close enough to its deadline.

Why `now - last >= period` and Not `now >= next`

The subtraction form

```
if (now - last >= 2000)
```

Survives the 32-bit counter wrapping after 49 days, because unsigned subtraction wraps consistently.

The comparison form

```
if (now >= next)
```

Breaks at the wrap: `next` is a huge number, `now` has reset to a small one, and the activity stops for 49 days.

A bug you cannot find by testing

It appears once, after seven weeks of continuous running. This is exactly the class of fault that reasoning catches and testing does not — which is why the argument matters more than the code.

Sharing Data With an ISR

Two hazards

- Missing `volatile` — the optimiser caches a value the ISR changes
- Torn access — `main()` reads a multi-word variable while the ISR is halfway through writing it

Three defences

- Mark it `volatile`
- Keep it to a single `uint32_t` — read atomically on a 32-bit machine
- Or disable interrupts briefly around the access

The flag pattern

The ISR does the minimum — clear the hardware flag, set a software one, return. `main()` does the real work. That keeps the handler short and keeps the shared state to one byte.

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 **The Complete Program**

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

The Complete Program (1) Shared state and interrupt handlers

```c
#include "stm32f4xx.h"

/* Shared state: written by ISRs, read by main */
volatile uint32_t msTicks     = 0;
volatile uint16_t adc_result  = 0;
volatile uint8_t  adc_ready   = 0;
volatile uint8_t  button_flag = 0;

/* Application state (main context only) */
static uint8_t show_millivolts = 0;

void SysTick_Handler(void) {
    msTicks++;      // its only job: advance time
}

void ADC_IRQHandler(void) {
    if (ADC1->SR & (1U << 1)) {   // EOC
        adc_result = (uint16_t)ADC1->DR;
        adc_ready  = 1;
    }
}
```

```c
void EXTI15_10_IRQHandler(void) {
    static uint32_t last_press = 0;

    if (EXTI->PR & (1U << 13)) {
        EXTI->PR = (1U << 13);  // clear FIRST

        // Debounce by timestamp,
        // no delay loop in an ISR.
        if ((msTicks - last_press) > 200) {
            last_press  = msTicks;
            button_flag = 1;
        }
    }
}
```

Each handler does the minimum: capture the event, set a `volatile` flag, return.

The Complete Program (2) Setup: LEDs and button

```c
static void leds_init(void) {
    RCC->AHB1ENR |= (1U << 1);   // GPIOB
    for (uint32_t pin = 0; pin < 4; pin++) {
        GPIOB->MODER &= ~(3U << (pin * 2));
        GPIOB->MODER |=  (1U << (pin * 2));
    }
}
```

Four outputs, PB0–PB3: the LED bar.

```c
static void button_init(void) {
    RCC->AHB1ENR |= (1U << 2);   // GPIOC
    GPIOC->MODER &= ~(3U << (13 * 2));
    GPIOC->PUPDR &= ~(3U << (13 * 2));
    GPIOC->PUPDR |=  (1U << (13 * 2));
    RCC->APB2ENR |= (1U << 14);  // SYSCFG
    SYSCFG->EXTICR[3] &= ~(0xFU << 4);
    SYSCFG->EXTICR[3] |=  (2U << 4);
    EXTI->FTSR |= (1U << 13);    // falling
    EXTI->IMR  |= (1U << 13);    // unmask
    NVIC_SetPriority(EXTI15_10_IRQn, 3);
    NVIC_EnableIRQ(EXTI15_10_IRQn);
}
```

Every peripheral follows the same shape: clock, pins, parameters, then use.

The Complete Program (3) Setup: ADC and UART

```
static void adc_init(void) {
    RCC->AHB1ENR |= (1U << 0);   // GPIOA
    RCC->APB2ENR |= (1U << 8);   // ADC1
    GPIOA->MODER |= (3U << (0 * 2));
    ADC1->SQR1  &= ~(0xFU << 20);
    ADC1->SQR3  &= ~(0x1FU << 0);
    ADC1->SMPR2 &= ~(7U << 0);
    ADC1->SMPR2 |=  (7U << 0);   // 480 cycles
    ADC1->CR1   |=  (1U << 5);   // EOCIE
    ADC1->CR2   |=  (1U << 0);   // ADON
    ADC1->CR2   |=  (6U << 24) | (1U << 28); // TIM2 TRGO
    NVIC_SetPriority(ADC_IRQn, 1);
    NVIC_EnableIRQ(ADC_IRQn);
}
```

ADC priority 1, button priority 3: the ADC is more urgent.

```
static void uart_init(void) {
    RCC->AHB1ENR |= (1U << 1);   // GPIOB
    RCC->APB2ENR |= (1U << 4);   // USART1
    GPIOB->MODER &= ~((3U<<(6*2)) | (3U<<(7*2)));
    GPIOB->MODER |=  ((2U<<(6*2)) | (2U<<(7*2)));
    GPIOB->AFR[0] &= ~((0xFU<<(6*4)) | (0xFU<<(7*4)));
    GPIOB->AFR[0] |=  ((7U<<(6*4)) | (7U<<(7*4)));
    USART1->BRR  = (104U << 4) | 3U; // 9600 @ 16 MHz
    USART1->CR1 |= (1U << 3) | (1U << 2); // TE, RE
    USART1->CR1 |= (1U << 13);   // UE
}
```

Pin map: `PA0` analog, `PB6`/`PB7` USART1, `PB0`–`PB3` LED bar, `PC13` button.

The Complete Program (4) Helpers

```
static void uart_write(uint8_t c) {
    while (!(USART1->SR & (1U << 7))) { }  // TXE
    USART1->DR = c;
}

static void uart_print(const char *s) {
    while (*s) uart_write((uint8_t)*s++);
}

static void uart_print_uint(uint32_t v) {
    char buf[11];
    int i = 0;
    if (v == 0) { uart_write('0'); return; }
    while (v) { buf[i++] = '0' + (v % 10); v /= 10; }
    while (i)  { uart_write((uint8_t)buf[--i]); }
}
```

```
// 12-bit reading -> 4-LED bar,
// written atomically via BSRR
static void leds_show_bar(uint16_t raw) {
    uint32_t level = (raw * 5U) / 4096U;  // 0..4
    uint32_t pattern = (level >= 4) ? 0x0F
                                    : ((1U << level) - 1U);
    GPIOB->BSRR = (pattern & 0x0F)
                | ((~pattern & 0x0F) << 16);
}
```

One BSRR store sets some LEDs and clears the others in the same cycle.

The Complete Program (5) `main()`: setup, the sampling timer, and the button task

```
int main(void) {
    __disable_irq();
    leds_init(); button_init();
    adc_init();  uart_init();
    SysTick_Config(16000000 / 1000);  // 1 ms tick
    __enable_irq();
    uart_print("\r\nSensor Monitor ready.\r\n");

    RCC->APB1ENR |= (1U << 0);   // TIM2: 10 Hz trigger
    TIM2->PSC = 1599;            // 16 MHz / 1600 = 10 kHz
    TIM2->ARR = 999;             // 10 kHz / 1000 = 10 Hz
    TIM2->CR2 = (2U << 4);       // TRGO on update
    TIM2->CR1 |= (1U << 0);      // start sampling
    while (1) {
        /* Task 1: none; TIM2 starts each conversion */

        /* Task 2: a result arrived (next slide) */

        /* Task 3: button pressed */
        if (button_flag) {
            button_flag = 0;
            show_millivolts = !show_millivolts;
            uart_print(show_millivolts
                       ? "-> millivolts\r\n" : "-> raw\r\n");
        }
    }
}
```

Read it as three `if`s

- Sampling: TIM2 starts each conversion in hardware.
- `adc_ready`: a result arrived.
- `button_flag`: the user pressed.

Each branch returns to the top. Even the slow UART branch cannot delay a sample.

The Complete Program (6) Task 2: a result arrived

```
if (adc_ready) {
    uint16_t raw;

    __disable_irq();            // critical section:
    raw       = adc_result;     // copy the shared value
    adc_ready = 0;
    __enable_irq();

    leds_show_bar(raw);

    if (show_millivolts) {
        // Integer maths only: no FPU needed.
        uint32_t mv = ((uint32_t)raw * 3300U) / 4095U;
        uart_print("Level: ");
        uart_print_uint(mv);
        uart_print(" mV\r\n");
    } else {
        uart_print("Raw:   ");
        uart_print_uint(raw);
        uart_print("\r\n");
    }
}
```

Why a critical section?

The ISR may fire between reading the value and clearing the flag. Interrupts are off for only three statements. The slow UART printing happens outside.

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 **Design Decisions Worth Noting**

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

Design Decisions Worth Noting

Decision	Why
Sampling neither drifts nor jitters	TIM2's update event starts each conversion in hardware. A software start would wait behind every 20 ms UART message, and the interval would wander. That breaks the uniform interval Nyquist relies on.
A minimal critical section	`adc_result` and `adc_ready` are read and cleared with interrupts off, so the ISR cannot overwrite the value between the two accesses. Only three statements are protected. UART printing happens outside.
Priorities follow urgency	ADC priority 1, button priority 3. A conversion result is lost if not read before the next one. A button press can wait.
Integer maths, no FPU	`(raw * 3300) / 4095`: multiply first to keep precision. 4095 $\times$ 3300 fits in 32 bits. Dividing first would truncate to zero.
The LED bar is atomic	One `BSRR` store updates all four LEDs, so no half-finished pattern is ever visible.

© Muním Zabidi

Suppose conversions were started in software by timed polling (Level 2). A student changes the line, and over a few minutes the rate falls below 10 Hz.

```
if (msTicks >= next_sample) {
    next_sample = msTicks + 100;   // <-- changed
    ADC1->CR2 |= (1U << 30);
}
```

Your prediction

- What is the interval between two samples?
- What does the extra delay add up to?
- What should the line be?

Suppose conversions were started in software by timed polling (Level 2). A student changes the line, and over a few minutes the rate falls below 10 Hz.

```
if (msTicks >= next_sample) {
    next_sample = msTicks + 100;   // <-- changed
    ADC1->CR2 |= (1U << 30);
}
```

Your prediction

- What is the interval between two samples?
- What does the extra delay add up to?
- What should the line be?

The answer

The interval is 100 ms *plus* however late the loop reached the test. That error accumulates. The correct form, `next_sample += 100;`, steps by a fixed amount, so lateness never adds up.

You Already Know This: A State Machine

This architecture is a state machine with a shared time base. You have designed exactly this, on paper, as a synchronous sequential circuit.

In digital electronics	In this program
Edge-triggered inputs	Interrupts that set flags
Combinational next-state logic	The `if` tests in the main loop
The clock	SysTick
State in flip-flops	State in `volatile` variables
Change behaviour: rewire	Change behaviour: edit the code

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 **The Timing Budget**

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

The Timing Budget

Write it before implementation, and it tells you whether the design is possible at all.

Activity	Period	Duration	Load
SysTick ISR	1 ms	2 µs	0.2 %
Sensor read	2 s	400 µs	0.02 %
Display update	100 ms	3 ms	3 %
Button ISR	rare	5 µs	≈ 0
Total			≈ 3.3 %

What the number tells you

3 % means comfortable headroom. 80 % means the design is fragile — one slow path and a deadline is missed. Over 100 % means it cannot work, and you know that before writing a line.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 **Predict, Then Measure**

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 Summary

Predict, Then Measure

The discipline

1. Predict the number — period, duty cycle, execution time
2. Measure it
3. When they disagree, find out why

How to measure

- Set a GPIO pin high on entry, low on exit — watch it on a scope
- Read `SysTick->VAL` before and after
- Toggle a pin per loop iteration and measure the frequency

The disagreement is the finding

If you predicted 400 µs and measure 3 ms, you have learned something — about the compiler, the clock tree, or your own assumptions.

Measuring without predicting teaches you nothing, because any number looks plausible.

© Muním Zabidi

Table of Contents

1. Why This Matters
2. From Requirement to Architecture
3. The Project: A Sensor Monitor
4. Architecture: The Non-Blocking Main Loop
5. The Complete Program
6. Design Decisions Worth Noting
7. The Timing Budget
8. Predict, Then Measure
9. **What a Working Demonstration Does Not Prove**
10. A Debugging Methodology
11. What You Have Learned
12. The Capstone Project
13. Summary

What a Working Demonstration Does Not Prove

It runs. What is still unknown?

- Whether it survives the counter wrap at 49 days
- Whether the deadline is met in the worst case, not the typical one
- Whether a shared variable is ever torn
- What happens when a sensor fails rather than reads wrong
- Whether it works at the temperature extremes

Demonstration is evidence, not proof

A demo shows the system works once, here, now, under the conditions you happened to create.

Everything else you believe about it comes from argument: the timing budget, the wrap analysis, the reasoning about shared state.

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 **A Debugging Methodology**

11 What You Have Learned

12 The Capstone Project

13 Summary

A Debugging Methodology

When a multi-peripheral system misbehaves, work bottom-up. Each step either finds the fault or eliminates a layer.

Step	Question
1	Is the clock enabled for this peripheral?
2	Is the pin in the right mode, with the right alternate function?
3	Does the register read back what you wrote?
4	Does the peripheral's status flag ever set?
5	Does the ISR run — toggle a pin inside it and look
6	Is the shared variable `volatile`, and read atomically?
7	Does the timing match the budget you predicted?

Steps 1 and 2 account for most first-time faults — and a register that does not read back is an unclocked peripheral.

© Muním Zabidi

Step	Question	The observation that settles it
1	Is the clock enabled?	Write `0xFFFF` to `GPIOA->MODER` and read it back. If it reads zero, the `RCC->AHB1ENR` bit is missing.
2	Is the pin an output?	Read `MODER` bits 11:10. If not `01`, the driver is disconnected.
3	Is `ODR` changing?	Watch `ODR` bit 5 while stepping. If it never changes, the write is not running, or `volatile` is missing.
4	Does the pin move?	Meter or scope on PA5. `ODR` toggles but the pin does not: an alternate function or analog mode has claimed it.
5	Is it the pin you think?	Check the schematic. On some Nucleo variants the LED is not on PA5.
6	Is the circuit right?	Voltage but no light: a reversed LED, an open resistor, or the wrong sink/source direction.

The walk goes up the five levels: register, configuration, C, pin voltage, circuit. Steps 1–4 use the debugger and a meter, not the source code. Rereading the program is almost always the slowest move.

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 **What You Have Learned**

12 The Capstone Project

13 Summary

What You Have Learned

The mechanism

- A CPU fetches, decodes, executes — forever
- Memory is numbered; some numbers are hardware
- C reaches them through pointers
- Instructions are what your C becomes

The practice

- Enable the clock first
- `volatile` on everything hardware touches
- Read–modify–write preserves the neighbours
- Interrupts for deadlines, timers for time
- Predict, then measure

The one sentence from Chapter 1

The CPU has no special instruction for “turn on an LED”. It has instructions that read and write addresses — and some of those addresses are wired to hardware.

Everything in the thirteen chapters since has been an elaboration of that.

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

10 A Debugging Methodology

11 What You Have Learned

12 **The Capstone Project**

13 Summary

Six weeks, in a group, on hardware you keep. It replaces a final exam question with a final exam *system*: one you can plug in, measure, and defend.

A shared core, then a free choice

Weeks 1–4: everyone builds the same core: GPIO with interrupt-driven input, and a UART debug link, under the non-blocking architecture.

Weeks 5–6: each group adds one enhancement peripheral, integrates it without breaking the core, and defends it.

Week	Deliverable
1	M1: GPIO output, verified with a meter or scope
2	M2: debounced EXTI input drives the output through a `volatile` flag
3	M3: UART debug link at a known baud rate
4	M4: integration in one non-blocking loop
5	Enhancement chosen, tested alone, integrated
6	Demonstration and written report

© Muním Zabidi

The Required Core By the end of Week 4, whatever the enhancement

1. At least one GPIO output, changed only through a register write the group can point to and explain.
2. At least one external input handled through EXTI, not by polling. The handler clears the flag, records the event, returns.
3. A UART link at a stated baud rate, reporting at least one state change to a PC terminal.
4. The whole system as a non-blocking cooperative loop: interrupts set flags, the main loop does the work, no branch blocks another.
5. A one-paragraph timing budget, produced *before* Milestone 4 is demonstrated.

Milestones 1–3 may use the peripherals separately. Milestone 4 needs them combined: one program, one main loop, one timing budget.

© Munim Zabidi

Milestone Rubrics (1) Each milestone is 10 points, weighted equally

M1: GPIO output

Pts	Criterion
4	Pin configured (clock, `MODER`) and verified with a meter or scope, not just “the LED lit”.
3	The group states from memory which bits were written and why.
3	Named masks, no magic numbers; clock enable before any write.

M2: Debounced EXTI input

Pts	Criterion
3	Interrupt configured (`SYSCFG`, edge, `IMR`, NVIC) and reliable.
3	Debounce by timestamp, not a blocking delay in the handler.
2	Handler clears the pending bit first, sets a `volatile` flag, returns.
2	Main loop drives the M1 output correctly over at least ten presses.

M3: UART debug link

Pts	Criterion
4	Correct baud register value for the stated clock, verified by clean text on a PC terminal.
3	TXE polled correctly: no missed or duplicated characters.
3	The group states the baud rate, its clock, and the arithmetic, from memory.

M4: Integration

Pts	Criterion
4	All three peripherals in one non-blocking loop, no branch blocking another.
2	Every ISR-shared variable is `volatile`, and the group says why.
2	A timing budget written *before* the demonstration.
2	Live fault injection: the group finds the fault with the bottom-up method, not by guessing.

© Muním Zabidi

Option A: DHT11 sensor. One open-drain data line with a timing-based protocol. It needs bit-banging with microsecond delays, and a reading takes about 20 ms. The group must decide: one blocking call scheduled as a rare low-priority task, or interrupt-driven phases. Justify it against the timing budget.

Option B: WS2812 LEDs. Each bit is a precisely timed pulse (about 1.25 µs), 24 bits per LED, no clock line. Generate it by cycle-counted bit-banging, or from a timer's PWM with DMA.

A group may propose another peripheral, with approval, if it brings a comparable integration challenge.

Enhancement rubric (20 points)

Pts	Criterion
6	Works alone, verified independently (scope, reference reading).
6	Integrated without breaking M4; the timing budget is re-measured, not assumed.
4	Integration timing cost estimated beforehand, checked afterward.
4	The group explains the hardest design decision, un-prompted.

© Muním Zabidi

Live demonstration

Fault injection, as in Milestone 4. The instructor may disconnect a wire, ask for a register value, or ask what happens if an input arrives during the enhancement's busiest moment.

Written report: at most four pages

- functional decomposition
- final pin assignment
- timing budget for the complete system
- verification plan: what was measured, with what instrument, and the pass/fail criterion

A demonstration with no timing budget, and a timing budget with no demonstration, both earn partial credit at best. The two claims are not substitutes for one another.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 From Requirement to Architecture

3 The Project: A Sensor Monitor

4 Architecture: The Non-Blocking Main Loop

5 The Complete Program

6 Design Decisions Worth Noting

7 The Timing Budget

8 Predict, Then Measure

9 What a Working Demonstration Does Not Prove

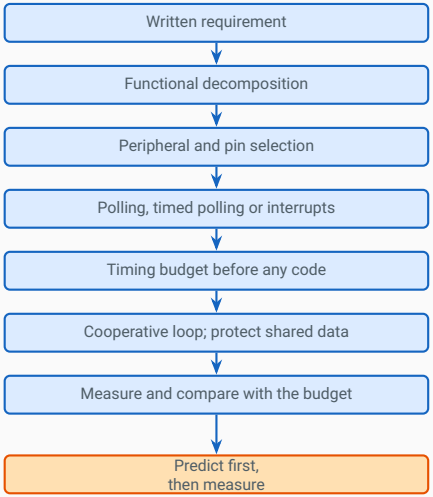
10 A Debugging Methodology

11 What You Have Learned

12 The Capstone Project

13 **Summary**

Summary



Terminology covered

- Functional decomposition
- Timing budget
- Cooperative main loop
- Shared data protection
- Debugging checklist

Last chapter

Use this method in every design you build.